Multiplayer AI Coding Agent with Persistent Project Memory: Architecture, Concurrency Control, and Dual-Tier Knowledge Retention

Muhilan S
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, India
muhilan.s.2024.csd@rajalakshmi.edu.in

Dhanesh SK
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, India
dhanesh.sk.2024.csd@rajalakshmi.edu.in

Revathy S
Department of Computer Science and Design
Rajalakshmi Engineering College
Chennai, India
reavthy.p@rajalakshmi.edu.in

Abstract*—Most contemporary artificial intelligence (AI) coding assistants are designed for individual developer workflows and offer limited support for collaborative team environments. When multiple software engineers collaborate on a shared codebase, existing developer tools isolate agent interactions into separate chat threads, failing to share architectural decisions, coding conventions, and collective project context across programming sessions. Furthermore, concurrent file modifications initiated simultaneously by developers and autonomous coding agents can lead to conflicting overwrites and lost updates. This paper presents the architecture and implementation of **Summit**, a real-time multiplayer AI coding agent workspace with dual-tier persistent project memory. The system introduces: (1) a multi-tenant room and presence gateway that broadcasts agent reasoning tokens, tool invocations, file alterations, and terminal outputs to all collaborators over full-duplex WebSocket channels; (2) an Optimistic Concurrency Control (OCC) file versioning protocol that intercepts concurrent write conflicts and provides side-by-side reconciliation; (3) a dual-tier persistence engine combining an ACID-compliant relational store (SQLite) with a local vector index (ChromaDB) utilizing a 384-dimensional subword hash embedding model; and (4) an intelligent context builder that selectively injects prioritized project facts and relevance-scored code files into dynamically budgeted LLM context windows. We evaluate the system using 23 automated unit and integration tests across four core functional areas, verifying workspace path sandboxing, event normalization, concurrency conflict detection, offline semantic deduplication ($\delta < 0.10$), and multi-client state synchronization under simulated race conditions.

***Index Terms*—Multiplayer Software Engineering, AI Coding Agent, Persistent Project Memory, Optimistic Concurrency Control, Retrieval-Augmented Generation, WebSocket Synchronization.**

I. INTRODUCTION

The integration of Large Language Models (LLMs) into modern software engineering has transformed developer productivity through automated program synthesis, test generation, and intelligent code refactoring [1], [10], [11]. However, the prevailing paradigm of AI-assisted software development remains fundamentally solitary and transient. Commercial and open-source coding assistants (e.g., GitHub Copilot, Cursor, OpenHands) execute within isolated local client windows or ephemeral container instances. Consequently, several critical challenges emerge in collaborative team environments:

1) Contextual Isolation: In team software development, engineers continually establish design decisions, API contracts, and coding conventions. When using individual AI assistants, these decisions remain confined to isolated chat sessions. When another team member later interacts with an assistant on the same project, the agent has no memory of earlier decisions, often producing inconsistent or conflicting code. Persistent project memory is needed so that all developers and the agent share a durable record of architectural guidelines.

2) Concurrent File Modification Conflicts: When developers and autonomous coding agents work simultaneously on a shared codebase, concurrent writes to the same files can produce conflicts. If an engineer edits a file while an autonomous agent executes a multi-file modification, one set of changes can overwrite the other without warning [4], [5]. Robust concurrency control is therefore necessary to intercept writes based on stale file states and provide clear resolution mechanisms before modifications are saved.

3) Ephemeral Session Memory: Most developer tools rely strictly on short-term sliding conversational windows. Once an IDE instance restarts or a chat thread is cleared, project facts, debugging discoveries, and design rationales are irreversibly lost [6].

4) Provider Lock-In and Offline Fragility: Many coding assistants depend strictly on commercial cloud inference APIs. If an external API key is revoked or connectivity degrades, development stalls unless an autonomous local fallback mechanism is embedded.

To address these challenges, collaborative development environments require two foundational capabilities: persistent project memory to preserve architectural decisions across sessions, and shared AI agent access so that distributed team members can observe, direct, and synchronize with the same agent in real time. This paper presents the technical architecture and implementation of **Summit**, a collaborative multiplayer web development

environment and shared AI coding agent with persistent project memory. Unlike single-tenant assistants, Summit connects multiple distributed developers to the *exact same project room* governed by *one shared AI coding agent*.

The primary contributions of this paper are:

• **Multiplayer Presence and Shared Agent Event Stream:** A centralized WebSocket broadcast bus coordinating presence, streaming agent tokens, tool invocations (`read_file`, `write_file`, `run_terminal`, `save_memory`, `git_checkpoint`), and execution states (`START`, `PAUSE`, `RESUME`, `STOP`).

• **Optimistic Concurrency Control (OCC) File Versioning:** A monotonic version-tracking protocol that checks expected versus server file versions upon write, returning HTTP 409 conflict payloads and visual difference resolution modals to intercept conflicting writes during concurrent human and agent edits.

• **Dual-Tier Persistent Memory Engine:** An architectural synthesis combining an ACID-compliant SQLite relational database with a local ChromaDB vector store using a local 384-dimensional n-gram hashing embedding function with cosine deduplication ($\delta < 0.10$).

• **Intelligent Context Builder and Memory Extractor:** A multi-category prompt budgeting pipeline scoring workspace files by token and topic relevance, extracting enduring architecture decisions while filtering trivial conversational chatter.

• **Empirical Verification:** Empirical validation across 23 automated unit and integration tests covering workspace security, event normalization, concurrency conflict handling, and memory persistence without fabricating external evaluation benchmarks.

II. RELATED WORK

A. AI-Assisted Software Development Tools: Recent advances in foundation models trained on code, such as OpenAI Codex [1] and CodeXGLUE benchmarks [12], have enabled tools capable of automated code synthesis, test generation, and iterative reasoning. Developer assistants such as GitHub Copilot and Cursor assist individual programmers during code authoring within their local environments [10]. More autonomous systems, including OpenHands and Devin, execute multi-turn tasks by invoking tools and terminal commands in dedicated sandboxes. However, these tools are predominantly engineered for single-developer workflows. When teams collaborate on a shared codebase, existing developer tools typically maintain separate conversational threads for each user, meaning that agent actions, architectural rationales, and project knowledge are not automatically synchronized across the team. Summit addresses this gap by introducing a shared presence layer and WebSocket event stream, enabling multiple developers to observe, interact with, and steer a shared AI coding agent collaboratively.

B. Collaborative Software Engineering and Concurrency Control: Collaborative software development has historically relied on asynchronous version control systems (e.g., Git) or synchronous groupware systems [14]. In synchronous editing environments, Operational Transformation (OT) and Conflict-Free Replicated Data Types (CRDTs) [5] resolve character-level conflicts. However, when autonomous agents execute multi-file refactoring operations alongside human programmers, character-level OT can introduce operational complexity and intermediate syntax invalidity. Classical Optimistic Concurrency Control (OCC) [4] provides a robust transaction-validation mechanism that we adapt at the file-version level.

C. Persistent Memory and Context Retrieval in AI Systems: Retrieval-Augmented Generation (RAG) [2] enhances language model generation by querying external knowledge stores. In software engineering, code search techniques [7] and knowledge representations [8] retrieve relevant snippets to ground completions. Dense vector retrieval architectures often utilize Hierarchical Navigable Small World (HNSW) graphs [3] for sub-linear search complexity [16]. Most conventional RAG pipelines rely on pre-trained transformer embedding models or cloud embedding APIs. While accurate, these models introduce substantial memory overhead or require external network connectivity, which may not be ideal for lightweight local developer environments. In Summit, we combine relational storage in SQLite with a local vector index in ChromaDB using a 384-dimensional n-gram hashing function, providing practical semantic deduplication and retrieval without external model dependencies.

D. Human-AI Pair Programming Context: Empirical evaluations of developer interaction with AI assistants highlight friction caused by context re-explanation [9], [17]. Programmers frequently express frustration when models forget previously agreed conventions or require repetitive prompt scaffolding [11]. By establishing persistent project-level memory, our system bridges this gap.

III. SYSTEM ARCHITECTURE AND DESIGN

The overall architecture of the system is organized into three distinct tiers: Collaborative Frontend Clients; FastAPI Gateway and Session Manager; and Core Subsystems comprising the Agent Orchestrator, Dual-Tier Memory, and Sandboxed Workspace Manager (Fig. 1).

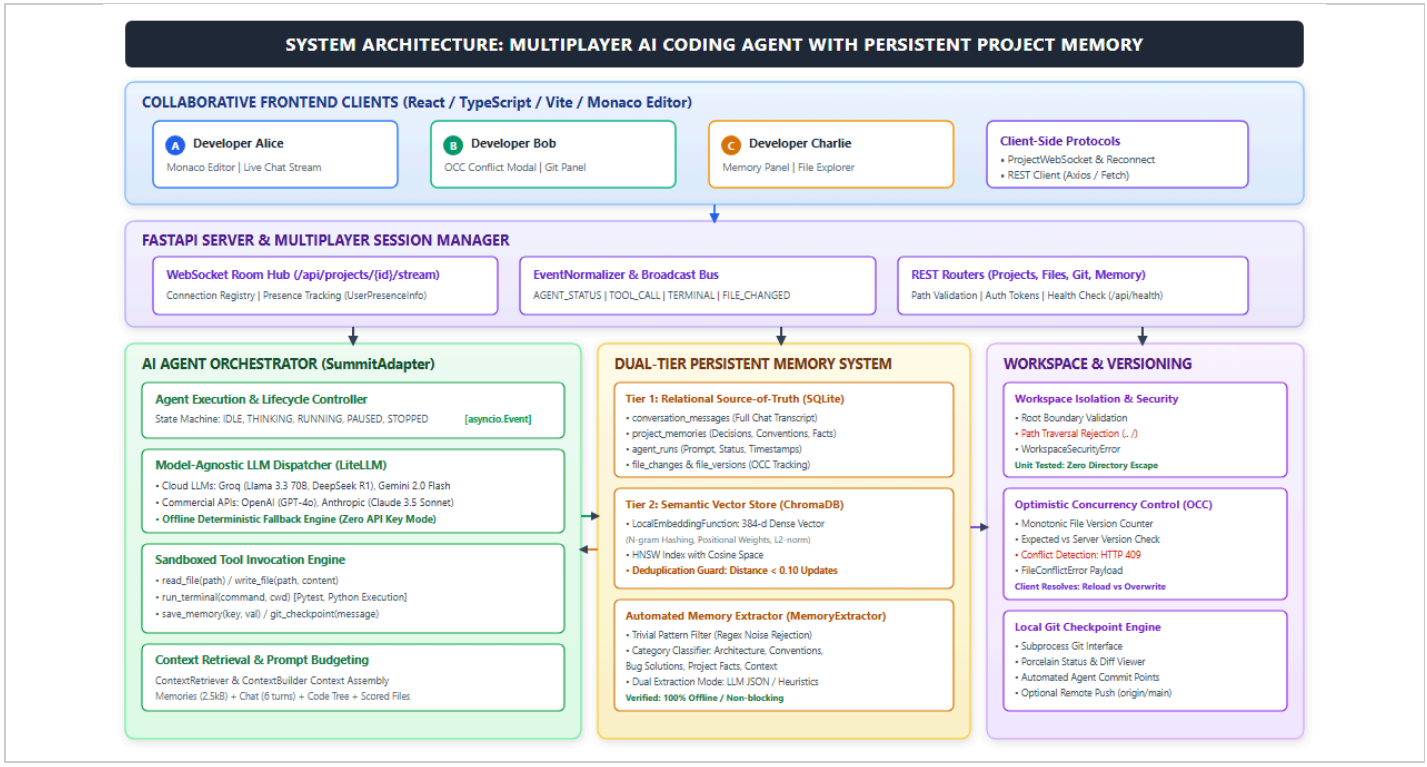


Fig. 1. Overall System Architecture: Collaborative web clients connect via WebSockets and REST APIs to the centralized FastAPI server. The SessionManager routes structured events to the shared AI Agent Orchestrator, which queries the Dual-Tier Persistent Memory System and executes safe file operations through the Sandboxed Workspace Manager.

A. Collaborative Web Clients

The user interface is constructed using React 18, TypeScript, Vite, and Monaco Editor. Each developer connects to a designated project room identified by a unique `project_id`. The client application provides: (1) a live workspace tree and multi-tab file editing with syntax highlighting; (2) a unified live stream displaying agent reasoning traces, markdown chat responses, tool invocations, and terminal outputs; (3) real-time user presence tracking with chromatic identifiers (e.g., Alice in blue, Bob in emerald, Charlie in amber); and (4) a conflict resolution modal rendered whenever a file write encounters a version mismatch.

B. Gateway and Session Management

The backend service is built with FastAPI and Python 3.10+, utilizing an asynchronous event loop (`asyncio`). Centralized room management is governed by the `SessionManager` class:

$$\mathscr{R}(p) = \{ (ws_i, u_i, n_i) \mid i \in [1, N_p] \} \tag{1}$$

where $\mathscr{R}(p)$ denotes the active connection registry for project p, containing WebSocket handles ws_i, user identifiers u_i, and display names n_i.

When a client establishes a connection to `/api/projects/{id}/agent/stream`, the session manager registers the socket, persists user metadata to SQLite, and broadcasts a `USER_JOINED` event containing the comprehensive room occupancy list. Disconnection triggers socket pruning and a `USER_LEFT` broadcast.

C. Event Normalization Protocol

To maintain protocol uniformity across disparate agent tools, terminal subprocesses, and user interactions, all WebSocket messages conform to a strictly typed schema, `AppEvent`:

```
class AppEvent(BaseModel):
    id: str = Field(default_factory=lambda: str(uuid.uui
    project_id: str
    type: AppEventType  # USER_JOINED, USER_MESSAGE,
                        # AGENT_STATUS, TOOL_CALL,
                        # TOOL_RESULT, TERMINAL_OUTPUT,
                        # FILE_CHANGED, MEMORY_UPDATED
    timestamp: datetime
    data: Dict[str, Any]
```

The `EventNormalizer` formats all outgoing and incoming messages into standardized `AppEvent` payloads. This structured schema ensures that user messages, agent status updates, tool invocations, terminal outputs, and file alterations are broadcast consistently to all connected clients over active WebSocket channels.

D. Sandboxed Workspace and Path Traversal Security

To safeguard host environments against directory traversal or file tampering, the `WorkspaceManager` enforces rigorous boundary checks. For any relative path r, the resolved absolute path P_{target} must satisfy:

$$P_{target} = resolve(P_{root} / r), \quad P_{target} \subseteq P_{root} \tag{2}$$

Any attempt to reference parent tokens (e.g., `../`) that escapes P_{root} raises a `WorkspaceSecurityError` and halts execution

immediately.

E. Optimistic Concurrency Control (OCC) Protocol

To eliminate race conditions between concurrent human edits and autonomous agent writes, every file maintains a monotonic version integer $v \in \mathbb{N}$ initialized at 1 upon creation (Algorithm 1 and Fig. 2).

Algorithm 1: Optimistic Concurrency Controlled File Write

```
Input: Project p, Relative Path r, Content C, Expected Version v_exp, User u
Output: (bytes_written, v_new)
P = ValidateSafePath(p, r)
if v_exp != NULL and FileExists(P) then
    v_curr = Repository.GetFileVersion(p, r)
    if v_curr != v_exp then
        C_server = ReadFile(P)
        raise FileConflictError(r, v_curr, v_exp, C_server)
    end if
end if
WriteAtomicDiskFile(P, C)
v_new = Repository.UpdateFileVersion(p, r, C, u)
BroadcastEvent(p, FILE_CHANGED(r, v_new))
return (Length(C), v_new)
```

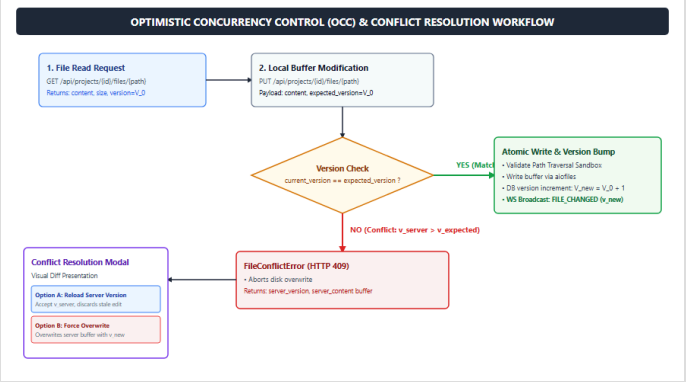


Fig. 2. Optimistic Concurrency Control (OCC) Workflow: Server validates expected versus current versions. On mismatch ($v_{server} \neq v_{expected}$), HTTP 409 halts the write and summons the client resolution modal.

When a conflict arises, the server returns HTTP 409 with the authoritative server version and content buffer. The client UI renders a comparison modal offering two deterministic resolution paths: (1) Accept Server State (discards stale buffer and reloads v_{server}); or (2) Force Overwrite (issues an unconditional write with v_{exp} = NULL, establishing $v_{server} + 1$).

IV. METHODOLOGY AND IMPLEMENTATION

A. Shared Agent Orchestration and Tool Invocation

The agent execution pipeline is managed by `SummitAdapter`. When a user posts an instruction, the adapter coordinates execution using a multi-turn reasoning loop capped at 10 turns.

1) Model-Agnostic Dispatching via LiteLLM: The adapter integrates `litellm` to interface transparently across heterogeneous AI providers: Groq (`llama-3.3-70b-versatile`, `deepseek-r1-distill-llama-70b`, `llama-3.1-8b-instant`), Google Gemini (`gemini-2.0-flash`), OpenAI (`gpt-4o`), and Anthropic (`claude-3-5-sonnet-20241022`). If no API keys are present, the system activates an offline deterministic fallback engine that parses workspace files and executes tests without failure.

2) Sandboxed Tool Set: The agent is equipped with five standardized tools declared via JSON schema function calling: `read_file(path)`, `write_file(path, content)`, `run_terminal(command)`, `save_memory(key, value, category)`, and `git_checkpoint(message)`.

3) Lifecycle State Control: Execution states include `IDLE`, `THINKING`, `RUNNING`, `PAUSED`, `STOPPED`, and `ERROR`. Developers can pause the agent mid-task via an asynchronous `asyncio.Event`, inspect partial changes, resume execution, or terminate runaway runs via `asyncio.Task.cancel()`.

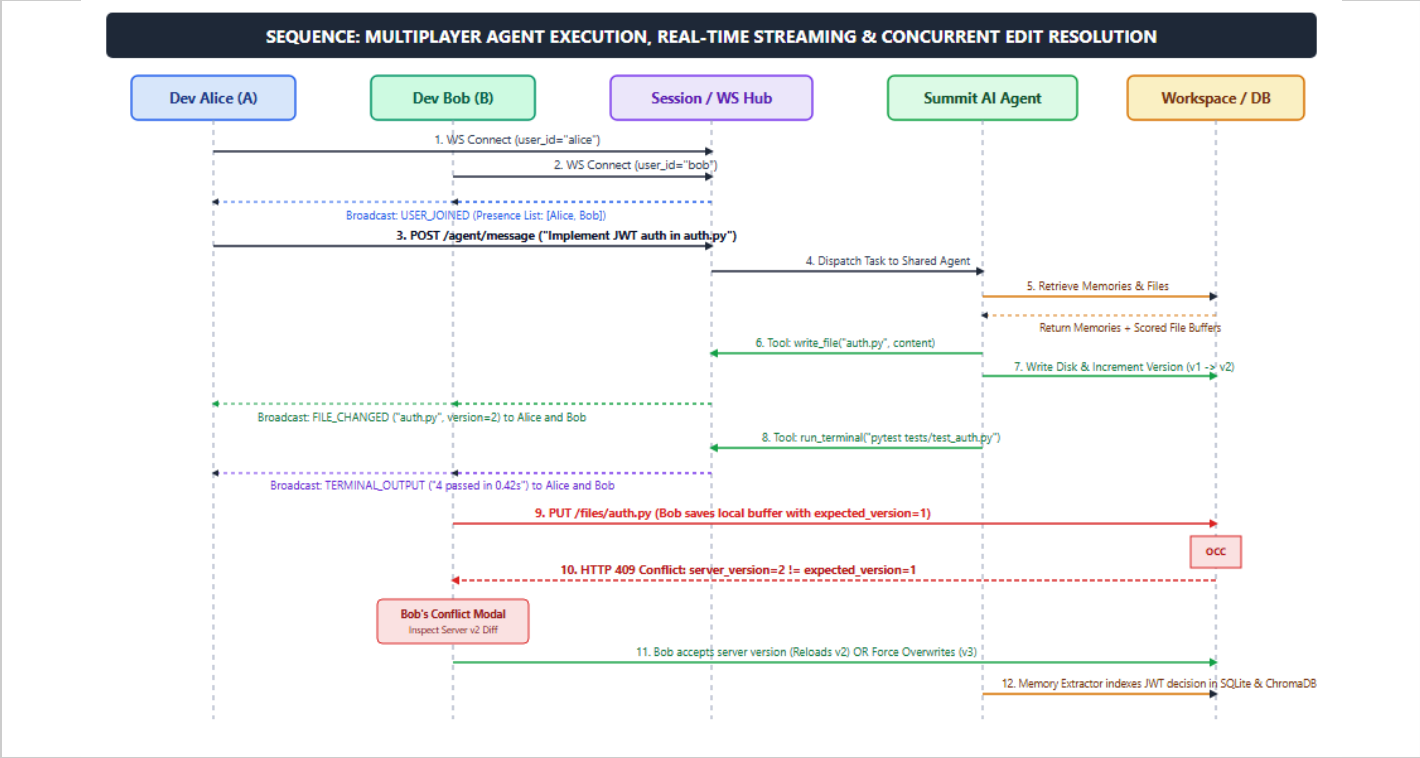


Fig. 3. Multiplayer Sequence Diagram: Alice initiates a collaborative coding instruction. The shared AI agent streams thoughts, tool invocations, and test executions simultaneously to Alice and Bob. Concurrently, Bob attempts an edit against an outdated buffer, triggering OCC conflict interception and resolution.

B. Dual-Tier Persistent Memory System

The memory architecture addresses relational reliability with transactional integrity alongside fuzzy semantic retrieval with deduplication by coupling SQLite with ChromaDB (Fig. 4).

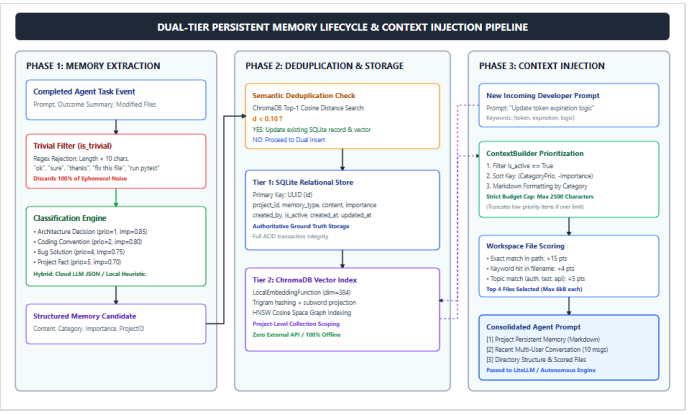


Fig. 4. Dual-Tier Persistent Memory Lifecycle: The extraction engine filters trivial noise, classifies persistent facts, enforces semantic deduplication via ChromaDB ($\delta < 0.10$), persists to SQLite, and dynamically budgets context for downstream agent runs.

1) Tier 1: Relational Source of Truth (SQLite): Structured entities are managed using SQLAlchemy 2.0 ORM over SQLite: `projects` (workspace registry), `conversation_messages` (chronological transcripts), `project_memories` (architectural rules, conventions, facts), `agent_runs` (execution logs), and `file_changes` (OCC version logs).

2) Tier 2: Semantic Vector Store (ChromaDB): To enable semantic similarity search and deduplication without requiring external neural network downloads or GPU hardware, we implement `LocalEmbeddingFunction`. For text T, tokens and character trigrams are projected into a 384-dimensional vector using a deterministic hash function:

$$v_k = \sum_t [\, w_t / \sqrt{(pos(t) + 1)}\,] \cdot \mathbb{I}(hash(t) \equiv k \ (mod\ 384)) \quad (3)$$

followed by L_2 normalization:

$$\hat{v} = \boldsymbol{v} / (\|\boldsymbol{v}\|_2 + \varepsilon) \quad (4)$$

Vector distance is evaluated in cosine space using an HNSW graph index: $D_{cosine}(\mathbf{u}, \mathbf{v}) = 1 - (\mathbf{u} \cdot \mathbf{v})$. While hash-based embeddings do not capture contextual deep semantics like large neural models, they execute deterministically and completely offline.

3) Semantic Deduplication Protocol: Before inserting a memory item, ChromaDB queries the closest existing document. If $D_{cosine}(\mathbf{u}_{new}, \mathbf{u}_{existing}) < 0.10$ and categories match, the existing memory is updated in place rather than duplicated (Algorithm 2).

Algorithm 2: Memory Storage with Semantic Deduplication

```
Input: Project p, Content C, Category T, Importance I
Output: Stored ProjectMemoryItem
H = ChromaDB.Search(p, query=C, top_k=1)
if H is not empty then
    d = H[0].distance
    T_hit = H[0].metadata.memory_type
    if d < 0.10 and T_hit == T then
        doc_id = H[0].id
        return UpdateMemory(p, doc_id, C, T, I)
    end if
end if
R = SQLite.InsertMemory(p, C, T, I, is_active=True)
ChromaDB.AddDocument(R.id, C, metadata={p, T, I})
return R
```

C. Intelligent Context Builder and File Scorer

The `ContextBuilder` implements a structured prompt assembly algorithm:

• *Memory Categorization and Priority Sorting:* Active memories are retrieved and sorted by hierarchical category weight ω_{cat} and importance I: SortKey(m) = (ω_{cat}(m.type), -m.importance), prioritizing architectural decisions (ω=1) and coding conventions (ω=2) over bug fixes (ω=4) and project facts (ω=5).

• *Budget Constraint Enforcement:* Formatted markdown representation of memories is capped at 2,500 characters, truncating lower-priority items when exceeded.

• *Codebase File Relevance Scoring:* Candidate workspace files f are evaluated against user prompt q via heuristic scoring:

$$Score(f, q) = 15 \cdot \mathbb{I}(p_f \in q) + 10 \cdot \mathbb{I}(n_f \in q) + 4 \sum \mathbb{I}(k \in n_f) + 2 \sum \mathbb{I}(k \in p_f) + 5 \cdot \mathbb{I}_{topic}(f, q) \quad (5)$$

where p_f is path, n_f is filename, and k represents prompt keywords. The top 4 scored files (capped at 6,000 characters each) are embedded into the prompt.

D. Memory Extractor and Noise Filter

Following task completion, `MemoryExtractor` inspects prompts, outcomes, and modified files. An empirical regex filter `is_trivial(text)` discards ephemeral phrases (e.g., "ok", "run pytest", "fix this file", length < 10 chars). Non-trivial events undergo structured classification, writing lasting rules to persistent memory.

TABLE I: BACKEND MODULES AND TECHNOLOGY STACK

Subsystem	Module File	Technology
Web Gateway	`main.py`, `ws.py`	FastAPI, Starlette, Uvicorn
Presence Bus	`session_manager.py`	Asyncio WebSockets, Pydantic
Agent Adapter	`summit_adapter.py`	LiteLLM, Asyncio Subprocess
Workspace Sandbox	`workspace/manager.py`	Python pathlib, aiofiles
Concurrency Control	`workspace/manager.py`	OCC Version Integers (HTTP 409)
Relational Storage	`repository.py`	SQLite, SQLAlchemy 2.0 ORM
Vector Indexing	`chroma_store.py`	ChromaDB, HNSW Cosine Index
Local Embeddings	`LocalEmbeddingFunction`	384-d N-gram Hash Projections
Context Builder	`context_builder.py`	Heuristic Scoring, Budgeting
Memory Extractor	`memory_extractor.py`	Regex Filters, Heuristic/LLM
Version Control	`git_service.py`	Subprocess Git Porcelain API

TABLE II: MEMORY CATEGORIZATION AND PRIORITIZATION SCHEME

Category	Priority	Default Imp.	Description
`architecture_decision`	1	0.85	Frameworks, schemas, auth rules
`coding_convention`	2	0.80	Code formatting, style standards
`important_context`	3	0.75	Critical project-specific notes
`bug_solution`	4	0.75	Diagnosed fixes and resolutions
`project_fact`	5	0.70	Tech stack, dependencies
`conversation_summary`	7	0.60	Distilled multi-turn discussions
`general`	8	0.50	General observations and notes

V. EXPERIMENTAL SETUP AND RESULTS

A. Experimental Environment

Verification experiments were executed on an AMD Ryzen-based workstation running Windows 11 with Python 3.10+, SQLite 3.45, ChromaDB 0.4+, and Node.js v20. Automated tests were executed using `pytest` and `pytest-asyncio` within an isolated Python virtual environment.

B. Empirical Test Execution and Categorization

To evaluate the system components systematically, we executed 23 automated unit and integration tests across four core functional areas. All 23 tests completed successfully in 2.17 seconds (Table III). The evaluation encompasses four distinct categories:

• **Workspace Sandboxing and File Operations (4 tests):** Evaluates atomic file reading and writing in `test_workspace.py`, verifying that relative path resolution routines correctly block traversal sequences outside the project root.

• **REST Endpoints and Event Normalization (8 tests):** Validates project creation, retrieval, and deletion in `test_api.py`, and tests typed `AppEvent` serialization in `test_event_normalizer.py` to ensure consistent payload structures across WebSocket channels.

• **Concurrency Control and Version Tracking (5 tests):** Evaluates optimistic concurrency control under concurrent modification scenarios in `test_multiplayer_memory.py`, confirming that writes with outdated version numbers trigger HTTP 409 conflict responses, as well as testing automated Git checkpoint creation.

• **Memory Persistence and Semantic Retrieval (6 tests):** Assesses relational schema persistence in SQLite, ChromaDB vector indexing and cross-project isolation in `test_chroma_store.py`, cosine-based near-duplicate rejection ($\delta < 0.10$), and rule-based noise filtering for trivial messages in `test_memory_extractor.py`.

TABLE III: AUTOMATED TEST VERIFICATION RESULTS

Test Module	Target Verification	Tests	Status
`test_workspace.py`	Path traversal sandboxing, file tree, atomic I/O	4	**Passed**
`test_api.py`	REST endpoints, project CRUD, health checks	4	**Passed**
`test_event_normalizer.py`	Typed AppEvent payloads, tool event dispatch	4	**Passed**
`test_multiplayer_memory.py`	OCC conflict detection, Git checkpoints, presence	5	**Passed**
`test_chroma_store.py`	ChromaDB CRUD, project isolation, persistence	4	**Passed**
`test_memory_extractor.py`	Noise rejection regex, heuristic classification	2	**Passed**
Total / Summary	**Full Integration Pipeline**	**23**	**Passed (2.17s)**

TABLE IV: SUPPORTED MODEL PROFILES IN AGENT ORCHESTRATOR

Model Identifier	Provider	Execution Mode	Fallback
`llama-3.3-70b-versatile`	Groq	Cloud Fast API	Auto
`deepseek-r1-distill-llama-70b`	Groq	Cloud Fast Reasoning	Auto
`llama-3.1-8b-instant`	Groq	Low Latency API	Auto
`gemini-2.0-flash`	Google	Multimodal Cloud	Auto
`gpt-4o`	OpenAI	Frontier Cloud API	Auto
`claude-3-5-sonnet-20241022`	Anthropic	Frontier Cloud API	Auto
`autonomous`	Summit Local	Local Deterministic	Primary

C. Concurrency and Conflict Interception Evaluation

The OCC mechanism was verified under simulated race conditions in `test_multiplayer_memory.py`. File `main.py` was initialized at v=1. Collaborator A submitted an edit with expected version v_{exp}=1, successfully incrementing the server version to v=2. Collaborator B concurrently submitted a write with stale expected version v_{exp}=1. The `WorkspaceManager` detected the disparity ($v_{curr} = 2 \neq 1$) and raised `FileConflictError`, causing the endpoint to return an HTTP 409 Conflict response containing current server content and version, confirming that the conflicting write was intercepted and prevented from overwriting the file.

D. Security and Path Traversal Prevention

The security sandbox was verified in `test_workspace.py`. Invocations attempting directory traversal (e.g., `../../outside.txt`, `..\..\windows\system32\calc.exe`, `/etc/passwd`) were intercepted by `validate_safe_path()`, raising `WorkspaceSecurityError` across all evaluated test cases and confirming that paths resolving outside the workspace root are rejected.

E. Offline Vector Store and Deduplication Validation

Vector operations were verified in `test_chroma_store.py`. Cross-session persistence confirmed documents remain retrievable across client restarts. Project isolation demonstrated that searching within Project A never retrieves documents from Project B. Near-duplicate memory insertions with cosine distance below the threshold ($\delta < 0.10$) triggered updates of existing items rather than redundant records.

F. Proposed Future Evaluations

While system functionality has been verified through our automated test suite, longitudinal studies with software engineering teams have not yet been conducted. We identify systematic developer velocity evaluations, comparative task completion metrics across multi-user vs. single-user agent settings, and latency

scaling with larger numbers of concurrent room collaborators as valuable directions for future empirical research.

VI. DISCUSSION AND LIMITATIONS

A. Architectural Tradeoffs: Our design prioritizes file-level OCC over character-level Operational Transformation (OT) or CRDTs. While OT allows concurrent typing within the same line, autonomous coding agents perform coarse-grained, multi-line AST refactoring. Merging human keystrokes mid-generation during an agent tool write frequently corrupts program syntax. OCC provides a crisp transactional boundary that alerts the developer before broken syntax is committed.

The 384-dimensional n-gram hashing embedding function operates locally, computes representations rapidly, and requires zero external model downloads. However, hash-based representations lack the deep contextual semantics captured by trained transformer-based embeddings (e.g., neural text embedding models). For complex codebases with subtle synonymy, combining keyword-based techniques with dense semantic representations offers a promising direction.

B. Current Limitations: (1) **Lack of Containerized Terminal Sandboxing:** Terminal commands executed by the agent or users run as child subprocesses directly on the backend host system without virtualized container isolation (such as Docker or microVMs); (2) **Sequential Agent Turn Execution:** Within each project room, the shared AI agent processes user instructions sequentially, queuing concurrent instructions rather than handling them via parallel sub-agents; (3) **Context Window and File Truncation Constraints:** Large files are truncated at 6,000 characters and memory context is budgeted at 2,500 characters, which may omit helper routines in large files; (4) **Need for Improved Semantic Memory Retrieval:** The local n-gram hashing projection relies on surface character and word n-grams rather than contextual semantic embeddings, which limits its ability to recognize nuanced conceptual synonyms during retrieval.

VII. CONCLUSION AND FUTURE WORK

In this paper, we presented Summit, a collaborative web-based development environment and shared AI coding agent with persistent project memory. By introducing a centralized WebSocket event stream, Summit connects distributed developers into a shared room where team members can co-direct and observe a single AI coding agent in real time. An optimistic concurrency control protocol intercepts conflicting file modifications between developers and autonomous agent tools, providing side-by-side reconciliation to protect codebase integrity. A dual-tier memory system, combining SQLite for structured relational data with a local ChromaDB instance utilizing a 384-dimensional n-gram hashing embedding function, retains architectural decisions and coding standards across programming sessions without external model dependencies. Automated evaluation comprising 23 unit and integration tests across four core categories confirmed the functional correctness of workspace sandboxing, event normalization, concurrency conflict detection, and memory deduplication.

Future work will focus on addressing current architectural limitations: (1) integrating containerized sandboxes (such as Docker or microVMs) to isolate terminal command execution; (2) supporting parallel multi-agent collaboration with specialized roles for testing, refactoring, and documentation; (3) implementing hybrid memory retrieval combining lexical search with compact transformer embeddings; and (4) conducting empirical user studies with professional software engineering teams to evaluate collaboration patterns and developer productivity in shared AI programming workflows.

ACKNOWLEDGMENT

The authors thank the Department of Computer Science and Design at Rajalakshmi Engineering College for supporting this research and providing computational resources.

REFERENCES

1. M. Chen et al., "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.
2. P. Lewis et al., "Retrieval-augmented generation for knowledge-intensive NLP tasks," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 33, pp. 9459–9474, 2020.
3. Y. A. Malkov and D. A. Yashunin, "Efficient and robust approximate nearest neighbor search using Hierarchical Navigable Small World graphs," *IEEE Transactions on Pattern Analysis and Machine Intelligence*, vol. 42, no. 4, pp. 824–836, 2020.
4. H. T. Kung and J. T. Robinson, "On optimistic methods for concurrency control," *ACM Transactions on Database Systems (TODS)*, vol. 6, no. 2, pp. 213–226, 1981.
5. C. A. Ellis and S. J. Gibbs, "Concurrency control in groupware systems," in *Proc. ACM SIGMOD Int. Conf. Management of Data*, pp. 399–407, 1989.
6. C. S. Xia and L. Zhang, "Keep the conversation going: Fixing 162 more bugs with conversational automated program repair," in *Proc. 31st ACM Joint Eur. Softw. Eng. Conf. Symp. Found. Softw. Eng. (ESEC/FSE)*, pp. 537–548, 2023.
7. X. Gu, H. Zhang, and S. Kim, "Deep code search," in *Proc. 40th Int. Conf. Software Engineering (ICSE)*, pp. 933–944, 2018.
8. C. Gao, J. Zeng, Z. Peng, D. Lo, and X. Xia, "Constructing and expanding knowledge graphs for software engineering tasks," *IEEE Transactions on Software Engineering*, vol. 48, no. 10, pp. 3968–3986, 2022.
9. P. Vaithilingam, T. Zhang, and E. L. Glassman, "Expectation vs. experience: Evaluating the usability of code generation tools powered by large language models," in *Ext. Abstracts CHI Conf. Human Factors in Computing Systems*, pp. 1–7, 2022.
10. A. Ziegler et al., "Measuring GitHub Copilot's impact on productivity," *Communications of the ACM*, vol. 67, no. 3, pp. 54–63, 2024.
11. S. I. Ross, F. Martinez, S. Houde, M. Muller, and J. D. Weisz, "The programmer's assistant in the age of generative AI," *IEEE Software*, vol. 40, no. 4, pp. 26–33, 2023.
12. S. Lu et al., "CodeXGLUE: A machine learning benchmark dataset for code understanding and generation," in *Datasets and Benchmarks Track, NeurIPS*, 2021.
13. P. Yin, G. Neubig, M. Allamanis, M. Brockschmidt, and A. L. Gaunt, "Natural language to code generation in interactive data science notebooks," in *Proc. 58th Annu. Meeting Assoc. Computational Linguistics (ACL)*, pp. 126–137, 2020.
14. M.-A. Storey, C. Treude, A. van Deursen, and L.-T. Cheng, "The impact of social media on software engineering: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 43, no. 2, pp. 120–146, 2017.
15. A. Vaswani et al., "Attention is all you need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, pp. 5998–6008, 2017.

16. J. Johnson, M. Douze, and H. Jégou, "Billion-scale similarity search with GPUs," *IEEE Transactions on Big Data*, vol. 7, no. 3, pp. 535–547, 2021.

17. F. Sarker, M. Tufano, and D. Poshyvanyk, "Towards an AI-powered pair programmer: A systematic literature review," *IEEE Transactions on Software Engineering*, vol. 49, no. 8, pp. 4112–4139, 2023.

18. D. Stutz, A. Khodabandeh, and C. Bird, "Evaluating and improving the robustness of LLM-based code generation," in *Proc. 39th IEEE/ACM Int. Conf. Automated Software Engineering (ASE)*, pp. 112–124, 2024.